

Trees & BST Solved Questions

Every problem from Part 8 of the Trees & BST guide, in order, with a worked C++ solution, the approach behind it, and the mistake that most often breaks it.

38

PROBLEMS

3

TIERS

14

PATTERNS

80

TESTS PASSING

Prepared for **Hari** · on-campus placement preparation

Companion to **Trees-and-BST-Guide.pdf** · code verified against **reference-part8-solutions.cpp**

Every solution in this document was compiled with **g++ -std=c++17 -O2 -Wall** and passes its test cases.

Generated 07 September 2026

Contents

Tier 1 — Learn the mechanics

- 01 LC 94 · Binary Tree Inorder Traversal
- 02 LC 144 · Binary Tree Preorder Traversal
- 03 LC 145 · Binary Tree Postorder Traversal
- 04 LC 102 · Binary Tree Level Order Traversal
- 05 LC 104 · Maximum Depth of Binary Tree
- 06 LC 111 · Minimum Depth of Binary Tree
- 07 LC 226 · Invert Binary Tree
- 08 LC 100 · Same Tree
- 09 LC 101 · Symmetric Tree
- 10 LC 112 · Path Sum
- 11 LC 543 · Diameter of Binary Tree
- 12 LC 110 · Balanced Binary Tree

Tier 2 — The placement core

- 13 LC 700 · Search in a Binary Search Tree
- 14 LC 701 · Insert into a Binary Search Tree
- 15 LC 450 · Delete Node in a BST
- 16 LC 98 · Validate Binary Search Tree
- 17 LC 230 · Kth Smallest Element in a BST
- 18 LC 235 · Lowest Common Ancestor of a BST
- 19 LC 236 · Lowest Common Ancestor of a Binary Tree
- 20 LC 108 · Convert Sorted Array to Binary Search Tree
- 21 LC 105 · Construct Binary Tree from Preorder and Inorder Traversal
- 22 LC 106 · Construct Binary Tree from Inorder and Postorder Traversal
- 23 LC 103 · Binary Tree Zigzag Level Order Traversal
- 24 LC 199 · Binary Tree Right Side View
- 25 LC 113 · Path Sum II
- 26 LC 129 · Sum Root to Leaf Numbers
- 27 LC 114 · Flatten Binary Tree to Linked List
- 28 LC 116 · Populating Next Right Pointers in Each Node

Tier 3 — Harder, but they do appear

- 29 LC 124 · Binary Tree Maximum Path Sum
- 30 LC 297 · Serialize and Deserialize Binary Tree
- 31 LC 437 · Path Sum III

- 32 LC 572 · Subtree of Another Tree
- 33 LC 863 · All Nodes Distance K in Binary Tree
- 34 LC 99 · Recover Binary Search Tree
- 35 LC 337 · House Robber III
- 36 LC 662 · Maximum Width of Binary Tree
- 37 LC 173 · Binary Search Tree Iterator
- 38 LC 1448 · Count Good Nodes in Binary Tree

Tier 1 — Learn the mechanics

Twelve problems. When you can write all twelve from a blank page — recursively and the traversals iteratively — you have the mechanics. Do not move on before that is true.

01 LC 94 · Binary Tree Inorder Traversal

EASY Pattern 1 — Traversal & collection

PROBLEM

Given the root of a binary tree, return the inorder traversal of its nodes' values — left subtree, then the node, then the right subtree. The follow-up asks for an iterative solution.

APPROACH

The recursive version is the three-line skeleton with the visit in the middle. The iterative version replaces the call stack with a real `stack`: dive as far left as you can while pushing every node, pop and visit, then step right and repeat.

The loop condition `cur || !st.empty()` matters — the moment you step right into a null child, the stack still holds unfinished ancestors, so an `!st.empty()` check alone would exit early.

Complexity $O(n)$ time · $O(h)$ space for the stack — $O(\log n)$ balanced, $O(n)$ skewed

C++ SOLUTION

```
class Solution {
public:
    // Recursive — left, NODE, right.
    vector<int> inorderTraversal(TreeNode* root) {
        vector<int> out;
        dfs(root, out);
        return out;
    }
    void dfs(TreeNode* n, vector<int>& out) {
        if (!n) return;
        dfs(n->left, out);
        out.push_back(n->val);
        dfs(n->right, out);
    }

    // Iterative — the follow-up the interviewer always asks for.
    vector<int> inorderIterative(TreeNode* root) {
        vector<int> out;
        stack<TreeNode*> st;
        TreeNode* cur = root;
        while (cur || !st.empty()) {
            while (cur) { st.push(cur); cur = cur->left; } // dive left, remembering the way back
            cur = st.top(); st.pop();
            out.push_back(cur->val); // visit
            cur = cur->right; // then go right
        }
        return out;
    }
};
```

```
    }  
};
```

WATCH OUT

Do not push a node twice. In the iterative version the inner `while` pushes on the way down and the outer loop pops exactly once, so each node is visited a single time. Remember why inorder matters: on a BST it emits values in sorted order, which is the key to LC 98, 230, 99 and 173.

02 LC 144 · Binary Tree Preorder Traversal

EASY Pattern 1 — Traversal & collection

PROBLEM

Return the preorder traversal — the node first, then the left subtree, then the right subtree.

APPROACH

Recursively, move the `push_back` to the top of the function. Iteratively, use a stack and push the **right** child first so the left one pops first — a stack reverses the order in which you push.

Preorder is the order you use whenever a node must be processed before its children: copying a tree, serialising one, or carrying state downward.

Complexity $O(n)$ time · $O(h)$ space

C++ SOLUTION

```
class Solution {
public:
    // Recursive — NODE, left, right.
    vector<int> preorderTraversal(TreeNode* root) {
        vector<int> out;
        dfs(root, out);
        return out;
    }
    void dfs(TreeNode* n, vector<int>& out) {
        if (!n) return;
        out.push_back(n->val);
        dfs(n->left, out);
        dfs(n->right, out);
    }

    // Iterative — push right first so left pops first.
    vector<int> preorderIterative(TreeNode* root) {
        vector<int> out;
        if (!root) return out;
        stack<TreeNode*> st; st.push(root);
        while (!st.empty()) {
            TreeNode* n = st.top(); st.pop();
            out.push_back(n->val);
            if (n->right) st.push(n->right);
            if (n->left) st.push(n->left);
        }
        return out;
    }
};
```

WATCH OUT

Pushing left before right in the iterative version silently gives you a mirrored traversal, which passes on a symmetric test tree and fails on every real one. Always test traversals on a lopsided tree.

03 LC 145 · Binary Tree Postorder Traversal

EASY Pattern 1 — Traversal & collection

PROBLEM

Return the postorder traversal — left subtree, right subtree, then the node.

APPROACH

Recursively, the `push_back` goes last. Iteratively there is a neat trick: run a preorder but push **left before right**, which produces node-right-left, then `reverse` the result. Reversing node-right-left gives left-right-node, which is exactly postorder.

Postorder is the order for anything where the parent's answer depends on the children's — heights, sizes, and safe deletion of a tree.

Complexity $O(n)$ time · $O(h)$ space, plus $O(n)$ for the output that gets reversed

C++ SOLUTION

```
class Solution {
public:
    // Recursive — left, right, NODE.
    vector<int> postorderTraversal(TreeNode* root) {
        vector<int> out;
        dfs(root, out);
        return out;
    }
    void dfs(TreeNode* n, vector<int>& out) {
        if (!n) return;
        dfs(n->left, out);
        dfs(n->right, out);
        out.push_back(n->val);
    }

    // Iterative — do preorder as node, right, left then reverse it.
    vector<int> postorderIterative(TreeNode* root) {
        vector<int> out;
        if (!root) return out;
        stack<TreeNode*> st; st.push(root);
        while (!st.empty()) {
            TreeNode* n = st.top(); st.pop();
            out.push_back(n->val);
            if (n->left) st.push(n->left);
            if (n->right) st.push(n->right);
        }
        reverse(out.begin(), out.end());
        return out;
    }
};
```

WATCH OUT

The honest two-stack or visited-flag postorder is fine too, but the reverse trick is shorter and less error-prone under interview pressure. Say out loud that you are building node-right-left and reversing it, otherwise it looks like you wrote preorder by mistake.

04 LC 102 · Binary Tree Level Order Traversal

MEDIUM Pattern 5 — Level-order / BFS

PROBLEM

Return the values of the nodes level by level, from left to right, as a vector of vectors — one inner vector per level.

APPROACH

BFS with a queue. The whole technique is one line: capture `int sz = q.size()` **before** the inner loop. At that instant the queue holds exactly the current level, so looping `sz` times processes precisely one level and the children you push land in the next round.

This template with a one-line edit inside the loop solves 103, 107, 199, 111, 515, 637, 662 and 116 — learn it once and reuse it everywhere.

Complexity $O(n)$ time · $O(w)$ space, where w is the widest level — up to $n/2$ for a perfect tree

C++ SOLUTION

```
class Solution {
public:
    vector<vector<int>> levelOrder(TreeNode* root) {
        vector<vector<int>> out;
        if (!root) return out;
        queue<TreeNode*> q; q.push(root);
        while (!q.empty()) {
            int sz = q.size();           // freeze the level width BEFORE pushing children
            vector<int> level;
            for (int i = 0; i < sz; i++) {
                TreeNode* n = q.front(); q.pop();
                level.push_back(n->val);
                if (n->left) q.push(n->left);
                if (n->right) q.push(n->right);
            }
            out.push_back(level);
        }
        return out;
    }
};
```

WATCH OUT

Reading `q.size()` inside the loop instead of freezing it beforehand mixes the children you just pushed into the current level, and every level smears into one. This is the single most common BFS bug in the topic.

05 LC 104 · Maximum Depth of Binary Tree

EASY

Pattern 2 — Aggregate over the whole tree

PROBLEM

Return the number of nodes along the longest path from the root down to the farthest leaf.

APPROACH

The purest instance of the universal skeleton. An empty tree has depth 0; otherwise the answer is one more than the deeper of the two children.

Every line maps onto a step of the model: the base case returns the identity value, the two recursive calls are trusted, and the combine step is `1 + max`.

Complexity $O(n)$ time · $O(h)$ space

C++ SOLUTION

```
class Solution {
public:
    int maxDepth(TreeNode* root) {
        if (!root) return 0; // empty tree: depth 0
        return 1 + max(maxDepth(root->left), maxDepth(root->right)); // me + the deeper child
    }
};
```

WATCH OUT

This problem measures depth in **nodes**, so an empty tree is 0 and a single node is 1. Diameter (LC 543) measures in **edges**. Mixing the two conventions produces answers that are off by exactly one, which is why it is worth stating which unit you are using before you write anything.

06 LC 111 · Minimum Depth of Binary Tree

EASY Pattern 2 — Aggregate, with the classic trap

PROBLEM

Return the number of nodes along the shortest path from the root down to the nearest **leaf**. A leaf is a node with no children at all.

APPROACH

This is not the mirror image of LC 104. A node with only one child would report a depth of 1 through its missing side, but a null child is not a leaf and therefore not a valid path end. So handle the single-child node explicitly: if one side is missing, the answer must come from the side that exists.

The BFS version is better in practice: return the depth of the **first leaf you dequeue**, which stops immediately instead of exploring a deep tree to the bottom.

Complexity DFS $O(n)$ time, $O(h)$ space · BFS $O(n)$ worst case but stops at the shallowest leaf

C++ SOLUTION

```
class Solution {
public:
    int minDepth(TreeNode* root) {
        if (!root) return 0;
        // A missing child is NOT a path of length 0 – it is no path at all,
        // so a node with one child must take the depth of the child it has.
        if (!root->left) return 1 + minDepth(root->right);
        if (!root->right) return 1 + minDepth(root->left);
        return 1 + min(minDepth(root->left), minDepth(root->right));
    }

    // BFS alternative: stop at the first leaf dequeued. Faster on wide, deep trees.
    int minDepthBFS(TreeNode* root) {
        if (!root) return 0;
        queue<TreeNode*> q; q.push(root);
        int depth = 1;
        while (!q.empty()) {
            int sz = q.size();
            for (int i = 0; i < sz; i++) {
                TreeNode* n = q.front(); q.pop();
                if (!n->left && !n->right) return depth;
                if (n->left) q.push(n->left);
                if (n->right) q.push(n->right);
            }
            depth++;
        }
        return depth;
    }
};
```

WATCH OUT

Writing `1 + min(minDepth(left), minDepth(right))` returns 1 on the tree `[1,2]` — wrong, the answer is 2. If a submission fails on a skewed input, this is almost always why.

07 LC 226 · Invert Binary Tree

EASY

Pattern 6 — Structural edit

PROBLEM

Invert the tree — mirror it about its vertical axis — and return the root.

APPROACH

Swap the two child pointers at every node and recurse. Order does not matter here: swap first and recurse, or recurse first and swap, both work, because the swap and the recursion touch disjoint things.

An iterative BFS version is equally acceptable — dequeue a node, swap its children, enqueue them.

Complexity $O(n)$ time · $O(h)$ space

C++ SOLUTION

```
class Solution {
public:
    TreeNode* invertTree(TreeNode* root) {
        if (!root) return nullptr;
        swap(root->left, root->right); // swap the pointers, not the values
        invertTree(root->left);
        invertTree(root->right);
        return root;
    }
};
```

WATCH OUT

Swap the **pointers**, not the values. Swapping `left->val` with `right->val` only exchanges two numbers and leaves the whole structure below them untouched, which passes on a two-node test and fails on anything deeper.

08 LC 100 · Same Tree

EASY

Pattern 6 — Two-tree lockstep recursion

PROBLEM

Given the roots of two binary trees, decide whether they are structurally identical and have the same values at every position.

APPROACH

Recurse on two nodes at once. The base cases enumerate the null combinations: both null means equal, exactly one null means unequal. After that, compare the values and recurse into the matching pairs.

This two-argument shape is the template for the whole comparison family — symmetric, subtree, merge, flip-equivalent.

Complexity $O(\min(n, m))$ time · $O(h)$ space

C++ SOLUTION

```
class Solution {
public:
    bool isSameTree(TreeNode* p, TreeNode* q) {
        if (!p && !q) return true;    // both empty -> equal
        if (!p || !q) return false;  // exactly one -> unequal
        return p->val == q->val
            && isSameTree(p->left, q->left)
            && isSameTree(p->right, q->right);
    }
};
```

WATCH OUT

Get the null cases in the right order: `if (!p && !q) return true;` must come **before** `if (!p || !q) return false;`, or two empty trees are reported as different.

09 LC 101 · Symmetric Tree

EASY Pattern 6 — Mirrored lockstep

PROBLEM

Decide whether a binary tree is a mirror image of itself about its centre.

APPROACH

This is LC 100 with the pairing crossed. Call a helper on `(root->left, root->right)`, and inside it compare `a->left` with `b->right` and `a->right` with `b->left`.

An empty tree is symmetric, so guard the root before calling the helper.

Complexity $O(n)$ time · $O(h)$ space

C++ SOLUTION

```
class Solution {
public:
    bool isSymmetric(TreeNode* root) {
        return !root || mirror(root->left, root->right);
    }
    bool mirror(TreeNode* a, TreeNode* b) {
        if (!a && !b) return true;
        if (!a || !b) return false;
        return a->val == b->val
            && mirror(a->left, b->right) // note the crossed pairing
            && mirror(a->right, b->left);
    }
};
```

WATCH OUT

The crossed pairing is the entire problem. Comparing `a->left` with `b->left` checks whether the two subtrees are *identical* rather than mirrored, which reports `[1,2,2,null,3,null,3]` as symmetric when it is not.

10 LC 112 · Path Sum

EASY

Pattern 4 — Root-to-leaf path

PROBLEM

Decide whether the tree has a root-to-leaf path whose node values add up to a given target.

APPROACH

Top-down: subtract the current value from the target on the way down, so by the time you reach a leaf the question is simply "is what remains zero?"

Decide **at the leaf**, not at null. Then return the OR of the two children — short-circuit evaluation means the right subtree is skipped as soon as the left one succeeds.

Complexity $O(n)$ time · $O(h)$ space

C++ SOLUTION

```
class Solution {
public:
    bool hasPathSum(TreeNode* root, int targetSum) {
        if (!root) return false; // no node, no path
        targetSum -= root->val;
        if (!root->left && !root->right) return targetSum == 0; // decide AT THE LEAF
        return hasPathSum(root->left, targetSum)
            || hasPathSum(root->right, targetSum);
    }
};
```

WATCH OUT

Testing `targetSum == 0` at a null node double-counts, because every leaf has two null children, and it also wrongly reports success on an empty tree. Two more edges: an empty tree is always `false`, and negative values mean you cannot prune a branch just because the running sum has already passed the target.

11 LC 543 · Diameter of Binary Tree

EASY Pattern 3 — Return one thing, record another

PROBLEM

Return the length of the longest path between any two nodes in the tree, measured in **edges**. The path need not pass through the root.

APPROACH

The longest path bending at a given node is `height(left) + height(right)`. But that is not what the parent needs — the parent needs this node's height. So return the height and record the diameter in a member variable as you go.

One postorder pass answers it. Say the sentence out loud while you write it: *I return the height, and I record the diameter.*

Complexity $O(n)$ time · $O(h)$ space

C++ SOLUTION

```
class Solution {
    int best = 0;
    int depth(TreeNode* n) {
        if (!n) return 0;
        int L = depth(n->left);
        int R = depth(n->right);
        best = max(best, L + R); // RECORD: longest path that bends at n (in edges)
        return 1 + max(L, R);   // RETURN: what my parent actually needs
    }
public:
    int diameterOfBinaryTree(TreeNode* root) {
        best = 0;
        depth(root);
        return best;
    }
};
```

WATCH OUT

Calling a separate `height()` function from inside a `diameter()` recursion recomputes heights at every node and is $O(n^2)$ — it times out on large inputs and is a common interview rejection. Also note the answer is in edges, so a two-node tree has diameter 1, not 2.

12 LC 110 · Balanced Binary Tree

EASY Pattern 3 — The poison-value trick

PROBLEM

Decide whether the tree is height-balanced: for every node, the heights of its two subtrees differ by at most one.

APPROACH

Same shape as diameter, with a twist that keeps it to a single pass: let the height function return `-1` to mean "something below me is already unbalanced", and propagate that poison value straight up without further work.

The top-level answer is then just `h(root) != -1`.

Complexity $O(n)$ time · $O(h)$ space

C++ SOLUTION

```
class Solution {
    // Returns the height, or -1 meaning "already unbalanced somewhere below".
    int h(TreeNode* n) {
        if (!n) return 0;
        int L = h(n->left); if (L == -1) return -1;
        int R = h(n->right); if (R == -1) return -1;
        if (abs(L - R) > 1) return -1;
        return 1 + max(L, R);
    }
public:
    bool isBalanced(TreeNode* root) { return h(root) != -1; }
};
```

WATCH OUT

The naive version — an `isBalanced` that calls `height` at every node — is $O(n \log n)$ on a balanced tree and $O(n^2)$ on a skewed one. Also note the condition must hold at **every** node, not only at the root: a tree can have balanced halves at the top and be badly unbalanced deeper down.

Tier 2 — The placement core

These are what actually get asked. LC 236, 98, 105 and 450 are the four most-asked tree questions in on-campus drives — be able to write them cold.

13 LC 700 · Search in a Binary Search Tree

EASY Pattern 9 — BST operations

PROBLEM

Given the root of a BST and a value, return the subtree rooted at the node with that value, or `nullptr` if it is not present.

APPROACH

Compare and descend: if the key is smaller go left, if larger go right, and stop when you land on it or run out of tree. Written iteratively it needs no extra space at all.

This compare-and-descend motion is the shape of the entire BST family — insert, delete, floor, ceil, closest value and LCA are all the same walk with a different action at each step.

Complexity $O(h)$ time · $O(1)$ space iteratively — $O(\log n)$ balanced, $O(n)$ skewed

C++ SOLUTION

```
class Solution {
public:
    // Iterative:  $O(h)$  time,  $O(1)$  space. The recursive version is equally accepted.
    TreeNode* searchBST(TreeNode* root, int val) {
        while (root && root->val != val)
            root = val < root->val ? root->left : root->right;
        return root; // nullptr if the key is absent
    }
};
```

WATCH OUT

Do not fall back on a full traversal. If a solution searches both subtrees it is $O(n)$ and throws away the only property that makes a BST worth having. State the $O(h)$ bound explicitly and note that it degrades to $O(n)$ on a skewed tree.

14 LC 701 · Insert into a Binary Search Tree

MEDIUM Pattern 9 — BST operations

PROBLEM

Insert a value into a BST and return the root. The value is guaranteed not to be present already, and any valid BST result is accepted.

APPROACH

Descend to the empty slot where the key belongs and hang a new node there. The recursive form is cleanest: the base case creates the node, and each level reassigns its child from the return value.

That *reassign-from-the-return-value* idiom is worth internalising here, because delete (LC 450) is built on exactly the same mechanism.

Complexity $O(h)$ time · $O(h)$ space recursive, $O(1)$ iterative

C++ SOLUTION

```
class Solution {
public:
    TreeNode* insertIntoBST(TreeNode* root, int val) {
        if (!root) return new TreeNode(val);           // the empty slot where val belongs
        if (val < root->val) root->left = insertIntoBST(root->left, val);
        else root->right = insertIntoBST(root->right, val);
        return root;                                   // reassign-from-return-value idiom
    }
};
```

WATCH OUT

Forgetting `return root;` at the end silently returns garbage and detaches the tree. And you must reassign — `insertIntoBST(root->left, val);` on its own, without `root->left =`, works only by accident for non-empty subtrees and loses the new node when the subtree is empty.

15 LC 450 · Delete Node in a BST

MEDIUM Pattern 9 — the three cases

PROBLEM

Delete the node with a given key from a BST and return the new root, keeping the BST property intact. The key may not exist.

APPROACH

Search down as usual. Once found, there are three cases. A leaf or a node with a single child is easy: return the child, which may be `nullptr`, and the parent's reassignment splices it in. A node with two children is the interesting one: copy in the value of its **inorder successor** — the smallest value in the right subtree — then delete that successor from the right subtree.

The successor works because it is the next value in sorted order, so putting it here keeps every ordering constraint on both sides intact. The inorder predecessor, the largest value in the left subtree, is equally valid.

Complexity $O(h)$ time · $O(h)$ space

C++ SOLUTION

```
class Solution {
public:
    TreeNode* deleteNode(TreeNode* root, int key) {
        if (!root) return nullptr;
        if (key < root->val) root->left = deleteNode(root->left, key);
        else if (key > root->val) root->right = deleteNode(root->right, key);
        else {
            // found it - three cases
            if (!root->left) return root->right; // leaf, or only a right child
            if (!root->right) return root->left; // only a left child
            TreeNode* succ = root->right; // two children: inorder successor
            while (succ->left) succ = succ->left; // = smallest value in the right subtree
            root->val = succ->val; // copy its value up
            root->right = deleteNode(root->right, succ->val); // then delete the successor
        }
        return root;
    }
};
```

WATCH OUT

After copying the successor's value you must delete the successor from the **right subtree**, not from the whole tree — deleting by value from the root would find the copy you just made and delete the wrong node. This is the most-asked BST question in on-campus interviews; write it from scratch at least twice.

16 LC 98 · Validate Binary Search Tree

MEDIUM Pattern 10 — the range method

PROBLEM

Decide whether a binary tree is a valid BST: every value in a node's left subtree is strictly smaller, every value in its right subtree strictly larger, and both subtrees are themselves BSTs.

APPROACH

Pass down the open interval each subtree is allowed to live in. Going left tightens the upper bound to the current value; going right tightens the lower bound. A node outside its interval fails immediately.

The alternative is an inorder walk checking that values strictly increase, keeping a `prev` pointer — same complexity, and it generalises to LC 99 and LC 530.

Complexity $O(n)$ time · $O(h)$ space

C++ SOLUTION

```
class Solution {
    // Every node must lie strictly inside the open interval (lo, hi).
    // Going left tightens hi; going right tightens lo.
    bool valid(TreeNode* n, long lo, long hi) {
        if (!n) return true;
        if (n->val <= lo || n->val >= hi) return false;
        return valid(n->left, lo, n->val) && valid(n->right, n->val, hi);
    }
public:
    bool isValidBST(TreeNode* root) {
        return valid(root, LONG_MIN, LONG_MAX); // long, because a node may hold INT_MIN/INT_MAX
    }
};
```

WATCH OUT

Checking only a node against its immediate parent is wrong: `[10,5,15,null,null,6,20]` passes that test and is not a BST, because 6 sits in 10's right subtree. And use `long` bounds — a node may legitimately hold `INT_MIN` or `INT_MAX`, which an `int` sentinel would reject.

17 LC 230 · Kth Smallest Element in a BST

MEDIUM Pattern 10 — early-stopping inorder

PROBLEM

Return the k-th smallest value in a BST, 1-indexed.

APPROACH

The inorder traversal of a BST is its sorted order, so the answer is the k-th value emitted. Decrement a counter at each visit and stop the moment it reaches zero, rather than materialising the whole sorted array.

For the follow-up — many queries on a tree that keeps changing — store a subtree size in each node. Then the k-th smallest is an $O(h)$ descent: compare k against `size(left) + 1` at each step.

Complexity $O(h + k)$ time · $O(h)$ space

C++ SOLUTION

```
class Solution {
    int k_ = 0, ans_ = -1;
    void inorder(TreeNode* n) {
        if (!n || k_ == 0) return;           // stop as soon as we have the answer
        inorder(n->left);
        if (k_ == 0) return;
        if (--k_ == 0) { ans_ = n->val; return; }
        inorder(n->right);
    }
public:
    int kthSmallest(TreeNode* root, int k) {
        k_ = k; ans_ = -1;
        inorder(root);                       // inorder of a BST is sorted order
        return ans_;
    }
};
```

WATCH OUT

The early return has to be checked in both places — once at the top of the function and once after the left recursion returns — otherwise the walk continues into the right subtree and overwrites the answer with a larger value.

18 LC 235 · Lowest Common Ancestor of a BST

MEDIUM

Pattern 7 — LCA by value

PROBLEM

Given a BST and two nodes present in it, return their lowest common ancestor. A node may be an ancestor of itself.

APPROACH

In a BST you never have to search. If both values are smaller than the current node, the answer lies entirely in the left subtree; if both are larger, in the right. The first node where they do **not** both go the same way is the split point, and that is the LCA.

Written as a `while` loop it is four lines and uses no extra space.

Complexity $O(h)$ time · $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    // In a BST the values tell you which way to go — no searching needed.  $O(h)$ ,  $O(1)$ .
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        while (root) {
            if (p->val < root->val && q->val < root->val) root = root->left;
            else if (p->val > root->val && q->val > root->val) root = root->right;
            else return root; // the split point (or one of p/q itself)
        }
        return nullptr;
    }
};
```

WATCH OUT

Do not reach for the general binary-tree LCA here — it works, but it is $O(n)$ and throws away the BST property the interviewer is testing for. Note also that the comparisons are by **value** in this problem, whereas LC 236 compares node **pointers**.

19 LC 236 · Lowest Common Ancestor of a Binary Tree

MEDIUM Pattern 7 — LCA

PROBLEM

Given a binary tree with no ordering property and two nodes that are both present, return their lowest common ancestor. A node may be an ancestor of itself.

APPROACH

Four lines, and worth memorising exactly. If the current node is null or is one of the two targets, return it. Otherwise recurse into both sides and read the results as "a target I found below me, or the answer": if **both** sides returned something, the two targets are on opposite sides and this node is the meeting point; if only one side did, pass that find upward.

The base case `root == p || root == q` is what makes the ancestor-of-itself case work without any special handling.

Complexity $O(n)$ time · $O(h)$ space

C++ SOLUTION

```
class Solution {
public:
    // Read the return value as "a target I found below me, or the answer".
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        if (!root || root == p || root == q) return root;
        TreeNode* L = lowestCommonAncestor(root->left, p, q);
        TreeNode* R = lowestCommonAncestor(root->right, p, q);
        if (L && R) return root;           // p and q are on opposite sides -> this is the LCA
        return L ? L : R;                 // both on one side -> pass the find upward
    }
};
```

WATCH OUT

This solution relies on both nodes actually being present. If one might be missing, it returns the other one instead of null, so you must add a verification pass — say so if the interviewer removes that guarantee. Also note the comparison is by pointer identity here, not by value.

20 LC 108 · Convert Sorted Array to Binary Search Tree

EASY Pattern 8 — construction

PROBLEM

Turn a sorted array into a height-balanced BST.

APPROACH

Take the middle element as the root and recurse on the two halves. Choosing the middle is exactly what forces balance — each side gets half the remaining elements, so the height is $O(\log n)$.

Build over index ranges rather than copying subarrays; slicing turns a linear algorithm into a quadratic one.

Complexity $O(n)$ time · $O(\log n)$ space for the recursion

C++ SOLUTION

```
class Solution {
    TreeNode* build(vector<int>& a, int lo, int hi) {
        if (lo > hi) return nullptr;
        int mid = lo + (hi - lo) / 2; // the middle becomes the root -> balanced
        TreeNode* n = new TreeNode(a[mid]);
        n->left = build(a, lo, mid - 1);
        n->right = build(a, mid + 1, hi);
        return n;
    }
public:
    TreeNode* sortedArrayToBST(vector<int>& nums) {
        return build(nums, 0, (int)nums.size() - 1);
    }
};
```

WATCH OUT

Write the midpoint as `lo + (hi - lo) / 2` rather than `(lo + hi) / 2` — the habit matters once indices are large enough to overflow. The base case is `lo > hi`, not `lo == hi`, which would drop every single-element range.

21 LC 105 · Construct Binary Tree from Preorder and Inorder Traversal

MEDIUM Pattern 8 — construction

PROBLEM

Given the preorder and inorder traversals of a binary tree with distinct values, reconstruct the tree.

APPROACH

Say the invariant out loud: **preorder gives the root, inorder says where it splits**. Take the next value from the preorder array as the root, find it in the inorder array, and everything to its left in the inorder is the left subtree, everything to its right is the right subtree.

Build a hash map from value to inorder index once, so each root lookup is $O(1)$. Consume the preorder array left to right with a single moving index, and build the left subtree before the right — that order is what keeps the index in step.

Complexity $O(n)$ time with the hash map · $O(n)$ space

C++ SOLUTION

```
class Solution {
    unordered_map<int,int> pos;    // value -> its index in inorder, so the split is  $O(1)$ 
    int pre = 0;
    TreeNode* build(vector<int>& preorder, int lo, int hi) {
        if (lo > hi) return nullptr;
        int rootVal = preorder[pre++];    // preorder gives the root...
        TreeNode* n = new TreeNode(rootVal);
        int mid = pos[rootVal];    // ...inorder says where it splits
        n->left = build(preorder, lo, mid - 1);
        n->right = build(preorder, mid + 1, hi);
        return n;
    }
public:
    TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
        pos.clear(); pre = 0;
        for (int i = 0; i < (int)inorder.size(); i++) pos[inorder[i]] = i;
        return build(preorder, 0, (int)inorder.size() - 1);
    }
};
```

WATCH OUT

Without the map, scanning the inorder array for each root makes it $O(n^2)$ on a skewed tree. And the preorder index must be a single shared cursor — passing a local copy down each branch quietly builds the wrong tree.

22 LC 106 · Construct Binary Tree from Inorder and Postorder Traversal

MEDIUM Pattern 8 — construction

PROBLEM

Given the inorder and postorder traversals of a binary tree with distinct values, reconstruct the tree.

APPROACH

Same idea as LC 105, read backwards. The **last** element of a postorder is the root, so walk the postorder array from the end. The element before the root belongs to the right subtree, so build the **right** child before the left one.

Everything else — the value-to-index map, the range recursion — is identical.

Complexity $O(n)$ time · $O(n)$ space

C++ SOLUTION

```
class Solution {
    unordered_map<int,int> pos;
    int post = 0;
    TreeNode* build(vector<int>& postorder, int lo, int hi) {
        if (lo > hi) return nullptr;
        int rootVal = postorder[post--]; // read postorder BACKWARDS
        TreeNode* n = new TreeNode(rootVal);
        int mid = pos[rootVal];
        n->right = build(postorder, mid + 1, hi); // RIGHT subtree first - it sits nearer the end
        n->left = build(postorder, lo, mid - 1);
        return n;
    }
public:
    TreeNode* buildTree(vector<int>& inorder, vector<int>& postorder) {
        pos.clear();
        post = (int)postorder.size() - 1;
        for (int i = 0; i < (int)inorder.size(); i++) pos[inorder[i]] = i;
        return build(postorder, 0, (int)inorder.size() - 1);
    }
};
```

WATCH OUT

Building the left subtree first here consumes the postorder cursor in the wrong order and produces a mirrored, wrong tree that still has the right node count. Worth knowing for interviews: preorder + postorder alone does **not** determine a tree uniquely unless every node has 0 or 2 children.

23 LC 103 · Binary Tree Zigzag Level Order Traversal

MEDIUM Pattern 5 — BFS variant

PROBLEM

Return the level-order traversal, but alternate direction: left to right on the first level, right to left on the second, and so on.

APPROACH

The LC 102 template with a boolean that flips each level. Rather than building the level and reversing it, size the vector up front and write each value straight into its final slot — index `i` when going left to right, `sz - 1 - i` when going right to left.

A deque-based version is equally valid: push front or back depending on the direction.

Complexity $O(n)$ time · $O(w)$ space

C++ SOLUTION

```
class Solution {
public:
    vector<vector<int>> zigzagLevelOrder(TreeNode* root) {
        vector<vector<int>> out;
        if (!root) return out;
        queue<TreeNode*> q; q.push(root);
        bool leftToRight = true;
        while (!q.empty()) {
            int sz = q.size();
            vector<int> level(sz);
            for (int i = 0; i < sz; i++) {
                TreeNode* n = q.front(); q.pop();
                // write straight into the right slot instead of reversing afterwards
                level[leftToRight ? i : sz - 1 - i] = n->val;
                if (n->left) q.push(n->left);
                if (n->right) q.push(n->right);
            }
            out.push_back(level);
            leftToRight = !leftToRight;
        }
        return out;
    }
};
```

WATCH OUT

Flip the direction flag once per level, at the end of the outer loop — flipping it inside the inner loop reverses per node and scrambles everything. Note the traversal itself never changes direction; only the way you record each level does.

24 LC 199 · Binary Tree Right Side View

MEDIUM Pattern 5 — BFS variant

PROBLEM

Standing to the right of the tree, return the values of the nodes you can see, ordered top to bottom.

APPROACH

The visible node on each level is the **last** one in level order, so run the LC 102 template and record the value when `i == sz - 1`.

A DFS alternative visits right before left and records the first node seen at each new depth — useful to mention, and it is the natural way to get the left view too.

Complexity $O(n)$ time · $O(w)$ space

C++ SOLUTION

```
class Solution {
public:
    vector<int> rightSideView(TreeNode* root) {
        vector<int> out;
        if (!root) return out;
        queue<TreeNode*> q; q.push(root);
        while (!q.empty()) {
            int sz = q.size();
            for (int i = 0; i < sz; i++) {
                TreeNode* n = q.front(); q.pop();
                if (i == sz - 1) out.push_back(n->val); // last node of the level
                if (n->left) q.push(n->left);
                if (n->right) q.push(n->right);
            }
        }
        return out;
    }
};
```

WATCH OUT

The rightmost visible node is not always the deepest right child — a long left branch can be the only node on its level. Taking `root->right` repeatedly is wrong; you must go level by level.

25 LC 113 · Path Sum II

MEDIUM Pattern 4 — paths with backtracking

PROBLEM

Return **all** root-to-leaf paths whose values sum to a given target, each as a list of node values.

APPROACH

LC 112 with the path recorded. Push the value on the way in, record a copy of the path when you hit a leaf with the target exhausted, and `pop_back` on the way out.

That `pop_back` is the backtracking step: it undoes this node's contribution so the sibling branch starts from the correct state. Passing the path by value instead removes the need for it, at the cost of copying the vector at every node.

Complexity $O(n)$ time to walk, up to $O(n \cdot h)$ to copy the results · $O(h)$ working space

C++ SOLUTION

```
class Solution {
    vector<vector<int>> out;
    vector<int> path;
    void dfs(TreeNode* n, int target) {
        if (!n) return;
        path.push_back(n->val);
        target -= n->val;
        if (!n->left && !n->right && target == 0) out.push_back(path); // record AT THE LEAF
        dfs(n->left, target);
        dfs(n->right, target);
        path.pop_back(); // BACKTRACK
    }
public:
    vector<vector<int>> pathSum(TreeNode* root, int targetSum) {
        out.clear(); path.clear();
        dfs(root, targetSum);
        return out;
    }
};
```

WATCH OUT

Forgetting `pop_back()` leaves nodes from other branches in the path, producing paths that do not exist. And record at the leaf, not at null — every leaf has two null children, so a null-based check emits each valid path twice.

26 LC 129 · Sum Root to Leaf Numbers

MEDIUM Pattern 4 — root-to-leaf

PROBLEM

Each root-to-leaf path spells a number by concatenating its digits, all values being 0–9. Return the sum of all such numbers.

APPROACH

Carry the number built so far downward: at each node the running value becomes `cur * 10 + node->val`. When you reach a leaf, that is the completed number — add it to the total.

A cleaner variant returns the sum instead of accumulating into a member, which avoids the reset-between-calls problem entirely.

Complexity $O(n)$ time · $O(h)$ space

C++ SOLUTION

```
class Solution {
    int total = 0;
    void dfs(TreeNode* n, int cur) {
        if (!n) return;
        cur = cur * 10 + n->val;           // build the number on the way down
        if (!n->left && !n->right) { total += cur; return; }
        dfs(n->left, cur);
        dfs(n->right, cur);
    }
public:
    int sumNumbers(TreeNode* root) {
        total = 0;
        dfs(root, 0);
        return total;
    }
};
```

WATCH OUT

Add to the total only at leaves. Adding at every node counts prefixes, and adding at null counts every path twice. If a member variable holds the total, reset it at the start of the public method — otherwise the second call on the same object returns nonsense.

27 LC 114 · Flatten Binary Tree to Linked List

MEDIUM Pattern 13 — in-place restructuring

PROBLEM

Flatten the tree in place into a "linked list" that uses the same `TreeNode` type: every `left` pointer is null and every `right` pointer leads to the next node in **preorder**.

APPROACH

The $O(1)$ -space solution is a Morris-style splice. At each node with a left subtree, find the rightmost node of that left subtree — the node that would be visited last within it — hang the current right subtree off it, move the left subtree across into `right`, and null out `left`. Then step right and repeat.

The recursive alternative flattens right first, then left, then rewires — it is easier to write but costs $O(h)$ stack.

Complexity $O(n)$ time · $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    // O(1) extra space: splice the left subtree in between the node and its right subtree.
    void flatten(TreeNode* root) {
        TreeNode* cur = root;
        while (cur) {
            if (cur->left) {
                TreeNode* pred = cur->left;
                while (pred->right) pred = pred->right; // rightmost node of the left subtree
                pred->right = cur->right; // old right tail hangs off it
                cur->right = cur->left; // left subtree moves across
                cur->left = nullptr; // and cut
            }
            cur = cur->right;
        }
    }
};
```

WATCH OUT

The pointer discipline from linked lists applies exactly: `cur->right = cur->left` before you have saved the old `cur->right` loses the entire right subtree. Splice first, then move, then cut.

28 LC 116 · Populating Next Right Pointers in Each Node

MEDIUM Pattern 13 — in-place restructuring

PROBLEM

In a **perfect** binary tree whose nodes have an extra `next` pointer, set each `next` to the node immediately to its right on the same level, or null if there is none.

APPROACH

A BFS solves it in $O(n)$ space by linking each dequeued node to the front of the queue. The $O(1)$ -space solution is better: once a level is linked, walk across it using the `next` pointers you just built, and use each node to link the level below — `n->left->next = n->right` within a parent, and `n->right->next = n->next->left` across two parents.

Because the tree is perfect, every node either has both children or none, which is what makes the two-line linking valid.

Complexity $O(n)$ time · $O(1)$ space

C++ SOLUTION

```
class Solution {
public:
    // Perfect tree, O(1) extra space: use the level you have already linked
    // to walk across and link the level below it.
    Node* connect(Node* root) {
        Node* leftmost = root;
        while (leftmost && leftmost->left) {
            for (Node* n = leftmost; n; n = n->next) {
                n->left->next = n->right;           // within the same parent
                if (n->next) n->right->next = n->next->left; // across two parents
            }
            leftmost = leftmost->left;
        }
        return root;
    }
};
```

WATCH OUT

This code relies on the tree being perfect. LC 117 drops that guarantee, and then you need a dummy-head plus tail pointer to assemble each level as you sweep — exactly the build-a-list-with-a-dummy-node technique from linked lists.

Tier 3 — Harder, but they do appear

Composite problems and the classic hard ones. Each is a pattern you already know, applied with one extra idea.

29 LC 124 · Binary Tree Maximum Path Sum

HARD Pattern 3 — clamp at zero

PROBLEM

A path is any sequence of nodes connected by edges; it need not pass through the root and may start and end anywhere. Return the maximum sum of node values along any path. Values may be negative.

APPROACH

The same return-one-record-another shape as diameter, with two adjustments for negatives. First, a child branch that sums to a negative number is worth dropping entirely, so clamp each child's contribution with `max(0, ...)`. Second, the value you return must be a **straight** path the parent can extend — the node plus its better single branch — while the value you record may **bend** at this node and use both branches.

Initialise the answer to `INT_MIN`, not 0, or an all-negative tree returns 0.

Complexity $O(n)$ time · $O(h)$ space

C++ SOLUTION

```
class Solution {
    int best = INT_MIN;
    int gain(TreeNode* n) {
        if (!n) return 0;
        int L = max(0, gain(n->left)); // a negative branch is worth dropping entirely
        int R = max(0, gain(n->right));
        best = max(best, n->val + L + R); // RECORD: the path that bends at n
        return n->val + max(L, R); // RETURN: a straight path the parent can extend
    }
public:
    int maxPathSum(TreeNode* root) {
        best = INT_MIN;
        gain(root);
        return best;
    }
};
```

WATCH OUT

Returning `n->val + L + R` to the parent is the classic error: a bent path cannot be extended upward without visiting a node twice. Return one branch, record two.

30 LC 297 · Serialize and Deserialize Binary Tree

HARD Pattern 11 — encoding

PROBLEM

Design an algorithm to encode a binary tree as a string and decode that string back into the identical tree.

APPROACH

Preorder with an explicit null marker. Serialising writes the value then recurses left and right, writing `#` for every null. Deserialising reads tokens in the same order: `#` returns null, anything else creates a node and recursively fills its left then right child.

It works because preorder plus null markers uniquely determines the structure — the recursion consumes exactly the tokens belonging to each subtree.

Complexity $O(n)$ time · $O(n)$ space for the string and $O(h)$ for the recursion

C++ SOLUTION

```
class Codec {
public:
    // Preorder with explicit '#' null markers — the markers are what make it decodable.
    string serialize(TreeNode* root) {
        string s;
        ser(root, s);
        return s;
    }
    void ser(TreeNode* n, string& s) {
        if (!n) { s += "#,"; return; }
        s += to_string(n->val) + ",";
        ser(n->left, s);
        ser(n->right, s);
    }

    TreeNode* deserialize(string data) {
        istringstream in(data);
        return des(in);
    }
    TreeNode* des(istringstream& in) {
        string tok;
        if (!getline(in, tok, ',')) return nullptr;
        if (tok == "#") return nullptr;
        TreeNode* n = new TreeNode(stoi(tok));
        n->left = des(in); // the recursion consumes the stream in the same order
        n->right = des(in);
        return n;
    }
};
```

WATCH OUT

Without the null markers the string is ambiguous: `1,2` could mean 2 is either child. Also keep the delimiter — values can be multi-digit and negative, so concatenating digits without commas is undecodable. For a **BST** specifically (LC 449) you can skip the markers, because the ordering recovers the structure.

31 LC 437 · Path Sum III

MEDIUM Pattern 4 + prefix-sum hash map

PROBLEM

Count the paths summing to a target where the path must go **downward** but may start and end at any node, not only the root and leaves.

APPROACH

This is subarray-sum-equals-k run along the DFS path. Keep a running sum from the root and a map counting how many times each prefix sum has been seen on the current path. At each node, the number of valid paths ending here is the number of times `run - target` has occurred.

Seed the map with `cnt[0] = 1` to represent the empty prefix — that is what lets a path starting at the root be counted.

Complexity $O(n)$ time · $O(h)$ space for the map along one path

C++ SOLUTION

```
class Solution {
    unordered_map<long long,int> cnt;    // prefix sum along the current path -> how many times seen
    int dfs(TreeNode* n, long long run, int target) {
        if (!n) return 0;
        run += n->val;
        int res = cnt.count(run - target) ? cnt[run - target] : 0;    // paths ending here
        cnt[run]++;
        res += dfs(n->left, run, target) + dfs(n->right, run, target);
        cnt[run]--;    // BACKTRACK the map as well as the path
        return res;
    }
public:
    int pathSum(TreeNode* root, int targetSum) {
        cnt.clear();
        cnt[0] = 1;    // the empty prefix - a path starting at the root
        return dfs(root, 0, targetSum);
    }
};
```

WATCH OUT

Decrement the map entry on the way out. Without that backtracking step, prefixes from one branch leak into a sibling branch and inflate the count. Use a 64-bit running sum — the values are large enough to overflow `int` on long paths, which is a hidden test case.

32 LC 572 · Subtree of Another Tree

EASY

Pattern 6 — comparison

PROBLEM

Decide whether `subRoot` appears as a subtree of `root` — a node of `root` together with **all** of its descendants must equal `subRoot` exactly.

APPROACH

Reuse `isSameTree` from LC 100 and try it at every node of the main tree. Straightforward and accepted.

The better answer, worth mentioning even if you code the simple one: serialise both trees preorder with null markers and delimiters, then check whether one string is a substring of the other. That is $O(n + m)$ with KMP.

Complexity $O(n \cdot m)$ time for the direct version, $O(n + m)$ via serialisation · $O(h)$ space

C++ SOLUTION

```
class Solution {
    bool same(TreeNode* a, TreeNode* b) {
        if (!a && !b) return true;
        if (!a || !b) return false;
        return a->val == b->val && same(a->left, b->left) && same(a->right, b->right);
    }
public:
    //  $O(n \cdot m)$ : try isSameTree at every node. The  $O(n+m)$  answer is to serialise
    // both trees with null markers and run a substring search — worth mentioning.
    bool isSubtree(TreeNode* root, TreeNode* subRoot) {
        if (!root) return !subRoot;
        if (same(root, subRoot)) return true;
        return isSubtree(root->left, subRoot) || isSubtree(root->right, subRoot);
    }
};
```

WATCH OUT

"Subtree" means the node **and everything below it** — a matching shape that continues with extra descendants does not count. In the serialisation version you must keep the null markers and a delimiter, or `12` matches inside `112` and you get false positives.

33 LC 863 · All Nodes Distance K in Binary Tree

MEDIUM Pattern 12 — tree as a graph

PROBLEM

Given a tree, a target node in it, and an integer k , return the values of all nodes exactly k edges away from the target — in any direction.

APPROACH

The difficulty is that distance runs upward as well as downward, and a tree has no parent pointers. So build them: one DFS recording each node's parent in a map. After that it is an ordinary BFS on a graph where each node has up to three neighbours — left, right and parent — with a visited set to stop you walking back.

When the BFS reaches depth k , everything still in the queue is exactly the answer.

Complexity $O(n)$ time · $O(n)$ space for the parent map and the visited set

C++ SOLUTION

```
class Solution {
    unordered_map<TreeNode*, TreeNode*> par;
    void mark(TreeNode* n, TreeNode* p) {
        if (!n) return;
        par[n] = p;
        mark(n->left, n);
        mark(n->right, n);
    }
public:
    // A tree has no upward edges, so build them, then it is an ordinary BFS on a graph.
    vector<int> distanceK(TreeNode* root, TreeNode* target, int k) {
        par.clear();
        mark(root, nullptr);
        unordered_set<TreeNode*> seen;
        queue<TreeNode*> q; q.push(target); seen.insert(target);
        int d = 0;
        while (!q.empty()) {
            if (d == k) {
                vector<int> out;
                while (!q.empty()) { out.push_back(q.front()->val); q.pop(); }
                return out;
            }
            int sz = q.size();
            for (int i = 0; i < sz; i++) {
                TreeNode* n = q.front(); q.pop();
                for (TreeNode* nb : {n->left, n->right, par[n]})
                    if (nb && !seen.count(nb)) { seen.insert(nb); q.push(nb); }
            }
            d++;
        }
        return {};
    }
};
```

WATCH OUT

The visited set is essential — without it the BFS bounces between a node and its parent forever. Handle `k = 0` (the answer is the target itself) and remember that `par[root]` is null, which the null check in the neighbour loop already covers.

34 LC 99 · Recover Binary Search Tree

MEDIUM Pattern 10 — inorder inversions

PROBLEM

Exactly two nodes of a BST were swapped by mistake. Recover the tree without changing its structure.

APPROACH

Walk inorder, which should produce strictly increasing values, and watch for inversions where the previous value exceeds the current one. Two swapped nodes produce **one** inversion if they are adjacent in the inorder sequence and **two** if they are not.

So: on the first inversion record `prev` as the first node; on every inversion record the current node as the second. Then swap the two values. Morris traversal gives the $O(1)$ -space follow-up.

Complexity $O(n)$ time · $O(h)$ space, or $O(1)$ with Morris

C++ SOLUTION

```
class Solution {
    TreeNode* first = nullptr, *second = nullptr, *prev = nullptr;
    void scan(TreeNode* n) {
        if (!n) return;
        scan(n->left);
        // Exactly two nodes are swapped: that is one inversion if they are adjacent
        // in the inorder sequence, two inversions if they are not.
        if (prev && prev->val > n->val) {
            if (!first) first = prev;
            second = n;
        }
        prev = n;
        scan(n->right);
    }
public:
    void recoverTree(TreeNode* root) {
        first = second = prev = nullptr;
        scan(root);
        if (first && second) swap(first->val, second->val);
    }
};
```

WATCH OUT

Set `first` only once — `if (!first) first = prev;` — but update `second` on every inversion. Overwriting `first` on the second inversion recovers the adjacent case and breaks the distant one, which is exactly the test that separates working solutions from nearly-working ones.

35 LC 337 · House Robber III

MEDIUM Pattern 14 — DP on trees

PROBLEM

Houses form a binary tree. Robbing two directly-connected houses alerts the police. Return the maximum amount that can be robbed.

APPROACH

Each node returns a pair of alternatives: the best total if this node is **skipped**, and the best if it is **robbed**. If skipped, each child is free to choose its own better option. If robbed, both children must be skipped.

The answer at the root is the larger of its two options. This tuple-return shape is the standard way to do DP on a tree — no memo table needed, because each node is visited once.

Complexity $O(n)$ time · $O(h)$ space

C++ SOLUTION

```
class Solution {
    // Returns {best if this node is skipped, best if this node is robbed}.
    pair<int,int> dfs(TreeNode* n) {
        if (!n) return {0, 0};
        auto L = dfs(n->left), R = dfs(n->right);
        int skip = max(L.first, L.second) + max(R.first, R.second); // children free to choose
        int take = n->val + L.first + R.first;                       // children must be skipped
        return {skip, take};
    }
public:
    int rob(TreeNode* root) {
        auto r = dfs(root);
        return max(r.first, r.second);
    }
};
```

WATCH OUT

The naive recursion — rob this node plus the four grandchildren, or skip it and take the children — recomputes the same subtrees repeatedly and is exponential without memoisation. The pair-return version is the same idea, computed once per node.

36 LC 662 · Maximum Width of Binary Tree

MEDIUM Pattern 5 — BFS with indices

PROBLEM

Return the maximum width of the tree, where the width of a level is the distance between its leftmost and rightmost non-null nodes, counting the null positions between them.

APPROACH

Index the nodes as if the tree were a complete array: a node at index i has children at $2i$ and $2i+1$. The width of a level is then $\text{lastIndex} - \text{firstIndex} + 1$.

Run the standard BFS carrying each node's index alongside it, and take the first and last index of every level.

Complexity $O(n)$ time · $O(w)$ space

C++ SOLUTION

```
class Solution {
public:
    int widthOfBinaryTree(TreeNode* root) {
        if (!root) return 0;
        unsigned long long best = 0;
        queue<pair<TreeNode*, unsigned long long>> q;
        q.push({root, 0});
        while (!q.empty()) {
            int sz = q.size();
            unsigned long long first = q.front().second, last = first;
            for (int i = 0; i < sz; i++) {
                auto [n, idx] = q.front(); q.pop();
                idx -= first; // rebase each level so the index cannot blow up
                last = idx;
                if (n->left) q.push({n->left, idx * 2});
                if (n->right) q.push({n->right, idx * 2 + 1});
            }
            best = max(best, last + 1);
        }
        return (int)best;
    }
};
```

WATCH OUT

The indices double every level, so on a deep skewed tree they overflow even 64-bit arithmetic. Rebase each level by subtracting its first index — the width is a difference, so shifting the whole level leaves the answer unchanged and keeps the numbers small. Using unsigned arithmetic makes the wrap-around harmless as well.

37 LC 173 · Binary Search Tree Iterator

MEDIUM Pattern 10 — a paused inorder walk

PROBLEM

Implement an iterator over a BST with `next()` returning the next smallest value and `hasNext()` reporting whether one remains. `next()` should run in average $O(1)$ time and the whole iterator in $O(h)$ memory.

APPROACH

This is the iterative inorder traversal from LC 94, frozen between calls. The constructor pushes the entire leftmost spine. `next()` pops the top — the smallest unvisited node — then pushes the leftmost spine of its right subtree. `hasNext()` is just whether the stack is non-empty.

The cost is amortised $O(1)$: each node is pushed once and popped once across the whole iteration, so n calls do $O(n)$ total work.

Complexity Amortised $O(1)$ per call · $O(h)$ space

C++ SOLUTION

```
class BSTIterator {
    stack<TreeNode*> st;
    void pushLeft(TreeNode* n) { while (n) { st.push(n); n = n->left; } }
public:
    // This is the iterative inorder traversal, paused between iterations.
    // Amortised O(1) per call, O(h) space.
    BSTIterator(TreeNode* root) { pushLeft(root); }

    int next() {
        TreeNode* n = st.top(); st.pop();
        pushLeft(n->right);
        return n->val;
    }

    bool hasNext() { return !st.empty(); }
};
```

WATCH OUT

Do not flatten the tree into a vector in the constructor — that is $O(n)$ memory and misses the point of the question. Say "amortised $O(1)$ " rather than " $O(1)$ ": a single `next()` can do $O(h)$ work, and it is the average across the traversal that is constant.

38 LC 1448 · Count Good Nodes in Binary Tree

MEDIUM

Pattern 14 — top-down counting

PROBLEM

A node is **good** if no node on the path from the root to it has a larger value. Count the good nodes in the tree.

APPROACH

Pure top-down: carry the maximum value seen so far on the path from the root. A node is good when its value is at least that maximum; then update the maximum and recurse.

Start with `INT_MIN` so the root always counts, which matches the definition — the root has nothing above it.

Complexity $O(n)$ time · $O(h)$ space

C++ SOLUTION

```
class Solution {
    int count = 0;
    // Top-down: carry the maximum seen on the path from the root down to here.
    void dfs(TreeNode* n, int mx) {
        if (!n) return;
        if (n->val >= mx) count++;
        mx = max(mx, n->val);
        dfs(n->left, mx);
        dfs(n->right, mx);
    }
public:
    int goodNodes(TreeNode* root) {
        count = 0;
        dfs(root, INT_MIN);
        return count;
    }
};
```

WATCH OUT

The comparison is `>=`, not `>` — a node equal to the running maximum still counts as good. And the maximum must be carried **down as a parameter**, not held in a member variable, or values from one branch leak into a sibling branch.